Program ROM

Test Report

Comprehensive Verification of PolyFlop8 Program ROM

Document ID:	TC-PROM-001
Revision:	1.0
Date:	January 5, 2026
Test Level:	Unit/Block Level
Test Engineer:	Artem Horiunov
Status:	**PASS**

Based on ISTQB Test Requirements Specification v1.0

1 TEST OVERVIEW

1.1 Unit Under Test (UUT)

Component	Program ROM for PolyFlop8 Processor
VHDL Entity	ProgPROM
Source File	ProgPROM.vhd, progprom_tb.vhd
Test Environment	ModelSim/QuestaSim 2023.4, VHDL Testbench
Address Width	11-bit (2048 addresses)
Data Width	16-bit

1.2 Scope

This test report documents the verification of the Program ROM for the PolyFlop8 Processor. The Program ROM is responsible for storing the program instructions and providing them to the CPU during the fetch cycle.

1.3 Test Objectives

- Verify correct operation of the Program ROM during instruction fetch
- Validate data integrity for predefined addresses
- Confirm asynchronous read functionality
- Ensure 100% code coverage of Program ROM behavioral description

2 REQUIREMENTS COVERAGE

Req ID	Requirement Description	Test Case ID	Status
TR-PROM-01	Verify fetch from address 0x000	TC-PROM-001-01	PASS
TR-PROM-02	Verify fetch from address 0x001	TC-PROM-001-02	PASS
TR-PROM-03	Verify fetch from address 0x002	TC-PROM-001-03	PASS
TR-PROM-04	Verify fetch from address 0x003	TC-PROM-001-04	PASS
TR-PROM-05	Verify fetch from address 0x004	TC-PROM-001-05	PASS
TR-PROM-06	Verify undefined address 0x005	TC-PROM-001-06	PASS
TR-PROM-07	Verify near max address 0x7FE	TC-PROM-001-07	PASS
TR-PROM-08	Verify max address 0x7FF	TC-PROM-001-08	PASS
TR-PROM-09	Verify asynchronous read	TC-PROM-001-09	PASS

Table 1: Requirements Coverage Matrix

3 DETAILED TEST CASES

3.1 Test Case TC-PROM-001-01: Fetch from Address 0x000

Requirement: TR-PROM-01
Purpose: Verify fetch from address 0x000.
Status: PASS

3.2 Test Case TC-PROM-001-02: Fetch from Address 0x001

Requirement: TR-PROM-02
Purpose: Verify fetch from address 0x001.
Status: PASS

3.3 Test Case TC-PROM-001-03: Fetch from Address 0x002

Requirement: TR-PROM-03
Purpose: Verify fetch from address 0x002.
Status: PASS

3.4 Test Case TC-PROM-001-04: Fetch from Address 0x003

Requirement: TR-PROM-04
Purpose: Verify fetch from address 0x003.
Status: PASS

3.5 Test Case TC-PROM-001-05: Fetch from Address 0x004

Requirement: TR-PROM-05
Purpose: Verify fetch from address 0x004.
Status: PASS

3.6 Test Case TC-PROM-001-06: Undefined Address 0x005

Requirement: TR-PROM-06
Purpose: Verify undefined address 0x005.
Status: PASS

3.7 Test Case TC-PROM-001-07: Near Max Address 0x7FE

Requirement: TR-PROM-07
Purpose: Verify near max address 0x7FE.
Status: PASS

3.8 Test Case TC-PROM-001-08: Max Address 0x7FF

Requirement: TR-PROM-08
Purpose: Verify max address 0x7FF.
Status: PASS

3.9 Test Case TC-PROM-001-09: Asynchronous Read Verification

Requirement: TR-PROM-09
Purpose: Verify asynchronous read functionality.
Status: PASS

4 SOURCE CODES

4.1 ProgPROM.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ProgPROM is
    Port (
        clk      : in  STD_LOGIC;
        reset    : in  STD_LOGIC;
        address  : in  STD_LOGIC_VECTOR (10 downto 0);
        data_out : out STD_LOGIC_VECTOR (15 downto 0)
    );
end ProgPROM;

architecture Behavioral of ProgPROM is

    constant OP_ADD : std_logic_vector(4 downto 0) := "00000";
    constant OP_SUB : std_logic_vector(4 downto 0) := "00001";
    constant OP_MOV : std_logic_vector(4 downto 0) := "01110";
    constant OP_JMP : std_logic_vector(4 downto 0) := "11000";

    type rom_type is array (0 to 2047) of std_logic_vector(15 downto 0);

    constant ROM : rom_type := (
        0 => "1000" & "001" & '0' & "00001010",
        1 => "1000" & "010" & '0' & "00000101",
        2 => OP_ADD & "001" & "010" & "00000",
        3 => OP_SUB & "001" & "010" & "00000",
        4 => OP_JMP & "00000000000",
        others => (others => '0')
    );

begin
    data_out <= ROM(to_integer(unsigned(address)));
end Behavioral;
```

4.2 progprom_tb.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity progprom_tb is
end progprom_tb;

architecture Behavioral of progprom_tb is

    component ProgPROM
        Port (
            clk      : in  STD_LOGIC;
            reset    : in  STD_LOGIC;
            address  : in  STD_LOGIC_VECTOR (10 downto 0);
            data_out : out STD_LOGIC_VECTOR (15 downto 0)
        );
    end component;

    signal clk_tb      : STD_LOGIC := '0';
    signal reset_tb    : STD_LOGIC := '0';
```

```
    signal address_tb  : STD_LOGIC_VECTOR(10 downto 0) := (others => '0');
    signal data_out_tb : STD_LOGIC_VECTOR(15 downto 0);

    constant CLK_PERIOD : time := 10 ns;

    function to_hex_string(slv : std_logic_vector) return string is
        variable hex_len : integer := (slv'length + 3) / 4;
        variable ret_str : string(1 to hex_len);
        variable nibble  : std_logic_vector(3 downto 0);
        variable padded_slv : std_logic_vector((hex_len * 4) - 1 downto 0);
    begin
        padded_slv := (others => '0');
        padded_slv(slv'length - 1 downto 0) := slv;
        for i in 0 to hex_len - 1 loop
            nibble := padded_slv((hex_len - i) * 4 - 1 downto (hex_len - i - 1)
                ↪ * 4);
            case nibble is
                when "0000" => ret_str(i+1) := '0';
                when "0001" => ret_str(i+1) := '1';
                when "0010" => ret_str(i+1) := '2';
                when "0011" => ret_str(i+1) := '3';
                when "0100" => ret_str(i+1) := '4';
                when "0101" => ret_str(i+1) := '5';
                when "0110" => ret_str(i+1) := '6';
                when "0111" => ret_str(i+1) := '7';
                when "1000" => ret_str(i+1) := '8';
                when "1001" => ret_str(i+1) := '9';
                when "1010" => ret_str(i+1) := 'A';
                when "1011" => ret_str(i+1) := 'B';
                when "1100" => ret_str(i+1) := 'C';
                when "1101" => ret_str(i+1) := 'D';
                when "1110" => ret_str(i+1) := 'E';
                when "1111" => ret_str(i+1) := 'F';
                when others => ret_str(i+1) := 'X';
            end case;
        end loop;
        return "0x" & ret_str;
    end function;

begin

    UUT: ProgPROM port map (
        clk      => clk_tb,
        reset    => reset_tb,
        address  => address_tb,
        data_out => data_out_tb
    );

    clk_process: process
    begin
        clk_tb <= '0';
        wait for CLK_PERIOD/2;
        clk_tb <= '1';
        wait for CLK_PERIOD/2;
    end process;

    stim_proc: process
    begin
        report "Starting␣PolyFlop8␣Program␣ROM␣Test␣Bench";
        reset_tb <= '1';
        wait for 20 ns;
        reset_tb <= '0';
        wait for 20 ns;
```

```
        report "Test␣Case␣1.1:␣Fetch␣from␣address␣0x000";
        address_tb <= std_logic_vector(to_unsigned(0, 11));
        wait for 10 ns;
        assert data_out_tb = x"820A"
            report "TC␣1.1␣Failed:␣Expected␣0x820A,␣got␣" & to_hex_string(
                ↪ data_out_tb)
            severity error;

        report "Test␣Case␣1.2:␣Fetch␣from␣address␣0x001";
        address_tb <= std_logic_vector(to_unsigned(1, 11));
        wait for 10 ns;
        assert data_out_tb = x"8405"
            report "TC␣1.2␣Failed:␣Expected␣0x8405,␣got␣" & to_hex_string(
                ↪ data_out_tb)
            severity error;

        report "Test␣Case␣1.3:␣Fetch␣from␣address␣0x002";
        address_tb <= std_logic_vector(to_unsigned(2, 11));
        wait for 10 ns;
        assert data_out_tb = x"0140"
            report "TC␣1.3␣Failed:␣Expected␣0x0140,␣got␣" & to_hex_string(
                ↪ data_out_tb)
            severity error;

        report "Test␣Case␣1.4:␣Fetch␣from␣address␣0x003";
        address_tb <= std_logic_vector(to_unsigned(3, 11));
        wait for 10 ns;
        assert data_out_tb = x"0940"
            report "TC␣1.4␣Failed:␣Expected␣0x0940,␣got␣" & to_hex_string(
                ↪ data_out_tb)
            severity error;

        report "Test␣Case␣1.5:␣Fetch␣from␣address␣0x004";
        address_tb <= std_logic_vector(to_unsigned(4, 11));
        wait for 10 ns;
        assert data_out_tb = x"C000"
            report "TC␣1.5␣Failed:␣Expected␣0xC000,␣got␣" & to_hex_string(
                ↪ data_out_tb)
            severity error;

        report "Test␣Case␣5.1:␣Undefined␣address␣0x005";
        address_tb <= std_logic_vector(to_unsigned(5, 11));
        wait for 10 ns;
        assert data_out_tb = x"0000"
            report "TC␣5.1␣Failed:␣Expected␣0x0000,␣got␣" & to_hex_string(
                ↪ data_out_tb)
            severity error;

        report "Test␣Case␣5.3:␣Near␣max␣address␣0x7FE";
        address_tb <= std_logic_vector(to_unsigned(2046, 11));
        wait for 10 ns;
        assert data_out_tb = x"0000"
            report "TC␣5.3␣Failed:␣Expected␣0x0000,␣got␣" & to_hex_string(
                ↪ data_out_tb)
            severity error;

        report "Test␣Case␣6.2:␣Max␣address␣0x7FF";
        address_tb <= (others => '1');
        wait for 10 ns;
        assert data_out_tb = x"0000"
            report "TC␣6.2␣Failed:␣Expected␣0x0000,␣got␣" & to_hex_string(
                ↪ data_out_tb)
```

```
            severity error;

        report "Test Case 2.2: Asynchronous read verification";
        wait until falling_edge(clk_tb);
        address_tb <= std_logic_vector(to_unsigned(0, 11));
        wait for 1 ns;
        assert data_out_tb = x"820A"
            report "Async Read Failed: Data did not update immediately for
                ↪ address 0x000"
            severity error;

        address_tb <= std_logic_vector(to_unsigned(1, 11));
        wait for 1 ns;
        assert data_out_tb = x"8405"
            report "Async Read Failed: Data did not update immediately for
                ↪ address 0x001"
            severity error;

        report "All PolyFlop8 Program ROM tests completed.";
        wait;
    end process;

end Behavioral;
```

5 SIMULATION LOG

```
Time resolution is 1 ps
Note: Starting PolyFlop8 Program ROM Test Bench
Time: 0 ps  Iteration: 0  Process: /progprom_tb/stim_proc  File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ProgProm_TB.vhd
Note: Test Case 1.1: Fetch from address 0x000
Time: 40 ns  Iteration: 0  Process: /progprom_tb/stim_proc  File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ProgProm_TB.vhd
Note: Test Case 1.2: Fetch from address 0x001
Time: 50 ns  Iteration: 0  Process: /progprom_tb/stim_proc  File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ProgProm_TB.vhd
Note: Test Case 1.3: Fetch from address 0x002
Time: 60 ns  Iteration: 0  Process: /progprom_tb/stim_proc  File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ProgProm_TB.vhd
Note: Test Case 1.4: Fetch from address 0x003
Time: 70 ns  Iteration: 0  Process: /progprom_tb/stim_proc  File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ProgProm_TB.vhd
Note: Test Case 1.5: Fetch from address 0x004
Time: 80 ns  Iteration: 0  Process: /progprom_tb/stim_proc  File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ProgProm_TB.vhd
Note: Test Case 5.1: Undefined address 0x005
Time: 90 ns  Iteration: 0  Process: /progprom_tb/stim_proc  File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ProgProm_TB.vhd
Note: Test Case 5.3: Near max address 0x7FE
Time: 100 ns  Iteration: 0  Process: /progprom_tb/stim_proc  File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ProgProm_TB.vhd
Note: Test Case 6.2: Max address 0x7FF
```

```
Time: 110 ns  Iteration: 0  Process: /progprom_tb/stim_proc  File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ProgProm_TB.vhd
Note: Test Case 2.2: Asynchronous read verification
Time: 120 ns  Iteration: 0  Process: /progprom_tb/stim_proc  File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ProgProm_TB.vhd
Note: All PolyFlop8 Program ROM tests completed.
Time: 122 ns  Iteration: 0  Process: /progprom_tb/stim_proc  File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ProgProm_TB.vhd
```

6 PASS/FAIL CRITERIA

6.1 Success Criteria

Criteria	Description
Functional	All test cases execute without assertion failures
Timing	No timing violations (Program ROM is combinational)
Coverage	100% statement coverage, 100% branch coverage achieved
Consistency	Identical results across multiple simulation runs
Documentation	All test results properly documented and signed off

6.2 Failure Conditions

- Any single assertion failure during simulation
- Code coverage less than 100% for Program ROM behavioral code
- Timing violations or simulation warnings
- Inconsistent results between simulation runs
- Missing or incomplete test documentation

6.3 Coverage Goals

- **Statement Coverage:** 100% of Program ROM process statements executed
- **Branch Coverage:** 100% of case statement branches taken
- **Condition Coverage:** All address decoding conditions tested
- **Toggle Coverage:** All signal bits toggled 0→1 and 1→0
- **FSM Coverage:** N/A (Program ROM is combinational)

7 SPECIAL TEST CONSIDERATIONS

7.1 Edge Cases

- Test all addresses with minimum and maximum values

- Verify proper handling of undefined addresses
- Test boundary conditions for address decoding
- Verify correct behavior during asynchronous read
- Test all possible multiplexer combinations

7.2 Timing Considerations

- Program ROM is combinational - no clock dependencies
- Monitor for glitches or hazards during input transitions
- Verify stable outputs within specified propagation delay
- Test with minimum, typical, and maximum delay models

7.3 Integration Considerations

- Verify interface compatibility with CPU and Control Unit
- Test handshake signals with Control Unit
- Validate output format matches processor expectations
- Confirm timing compatibility with other processor components